

Especificação do Documento de Realização de Caso de Uso

Projeto: Poupa+

Versão:1.1

Outubro de 2025

Equipe de desenvolvimento

Sumário

1. Introdução	3
1.1 Propósito	4
1.2 Escopo	4
1.3 Visão Geral de Arquitetura	4
2. Caso de Uso 01 - Registrar Gasto	4
2.1 Descrição do Caso de Uso	4
2.2 Diagrama de Classes Participantes	6
3. Caso de Uso 02 - Consultar Saldo	6
3.1 Descrição do Caso de Uso	6
3.2 Diagrama de Classes Participantes	7
4. Caso de Uso 03 - Cadastrar Categoria	7
4.1 Descrição do Caso de Uso	7
4.2 Diagrama de Classes Participantes	9
5. Caso de Uso 04 - Autenticar (Login)	9
5.1 Descrição do Caso de Uso	9
5.2 Diagrama de Classes Participantes	10
6. Caso de Uso 05 - Gerar Relatório/Sumário	10
6.1 Descrição do Caso de Uso	10
6.2 Diagrama de Classes Participantes	11
7. Observações Técnicas e Considerações de Implementação	11

Introdução

Propósito

Uma realização de caso de uso descreve como um caso de uso é executado no modelo de design, em termos de objetos de colaboração. Este documento organiza os artefatos relacionados a cada caso de uso (descrições, classes participantes, fluxos de colaboração e diagramas de sequência textuais), servindo como ponte entre requisitos funcionais e o design que será implementado.

Escopo

Este documento contém as realizações de casos de uso do sistema de Controle de Finanças Pessoais. Serão detalhados os seguintes casos de uso:

- Caso de Uso 01 - Registrar Gasto
- Caso de Uso 02 - Consultar Saldo
- Caso de Uso 03 - Cadastrar Categoria
- Caso de Uso 04 - Autenticar (Login)
- Caso de Uso 05 - Gerar Relatório/Sumário

Visão Geral de Arquitetura

O sistema prevê uma arquitetura em camadas: apresentação (UI), serviço/aplicação (APIs), domínio (classes: Usuario, Login, Saldo, Categoria, Gasto) e persistência (banco de dados). A autenticação é tratada no nível de serviço; operações de transação (registrar gasto, atualizar saldo) devem ocorrer em contexto transacional para garantir consistência.

Caso de Uso 01 - Registrar Gasto

Descrição do Caso de Uso

Objetivo: Permitir que o usuário registre uma nova despesa, associando-a a uma categoria e atualizando automaticamente o saldo. Pré-condição: Usuário autenticado. Pós-condição: Gasto persistido, categoria associada, saldo atualizado.

Classes Participantes

- **Usuario** — inicia a operação e possui permissões.
- **Login** — garante autenticação (pré-condição).
- **Gasto** — entidade que representa a despesa a ser registrada.
- **Categoria** — classifica o gasto.
- **Saldo** — responsável por recalcular o valor disponível do usuário.
- **RepositorioGasto** (ou DAO) — persiste o gasto.
- **ServicoSaldo** — lógica de cálculo/atualização do saldo.

Descrição da Colaboração (Fluxo Principal)

1. **Usuario** solicita a operação "Registrar Gasto" (interface/cliente).
2. Sistema valida sessão via **Login**/mecanismo de autenticação.
3. Interface solicita dados: valor, data, categoria, descrição.
4. **Usuario** submete os dados.
5. Controlador / Serviço aplica validações (valor > 0, categoria existe).
6. Serviço cria instância **Gasto** com os dados.
7. **RepositorioGasto** persiste o **Gasto**.
8. **ServicoSaldo** atualiza o **Saldo** do **Usuario** subtraindo o valor do gasto.

9. Sistema confirma operação e retorna sucesso ao **Usuario**.

Fluxos Alternativos

- A1: Categoria não encontrada → sistema solicita criação/seleção de categoria.
- A2: Valor inválido (nulo/<=0/format) → validação falha e erro retornado.
- A3: Falha de persistência → operação abortada, transação revertida, mensagem de erro.

Diagrama de Sequência (descrição textual)

```
Usuario → Sistema/UI : solicitarRegistrarGasto()
Sistema/UI → ServicoAutenticacao(Login) : validarSessao()
ServicoAutenticacao → ServicoGasto :
iniciarRegistroGasto(dados)
ServicoGasto → RepositorioCategoria : obterCategoria(id)
RepositorioCategoria → ServicoGasto : categoria
ServicoGasto → RepositorioGasto : salvar(gasto)
RepositorioGasto → ServicoGasto : confirmacaoSalvar
ServicoGasto → ServicoSaldo : atualizarSaldo(usuarioId,
-valor)
ServicoSaldo → RepositorioSaldo : persistir(saldoAtualizado)
ServicoSaldo → ServicoGasto : confirmacaoSaldo
ServicoGasto → Sistema/UI : resultadoSucesso()
Sistema/UI → Usuario : exibirMensagemSucesso()
```

Regras de Negócio

- Usuário deve estar autenticado.
- Valor do gasto deve ser positivo.
- Todo gasto deve estar vinculado a uma única categoria.
- Atualização de saldo ocorre em transação atômica com a persistência do gasto.

Resultado Esperado

Gasto persistido, saldo atualizado e usuário notificado.

Diagrama de Classes Participantes

(Adicionar no Astah: classes **Usuario**, **Login**, **Gasto**, **Categoria**, **Saldo**, **RepositorioGasto**, **ServicoSaldo** e relações já definidas no modelo.)

Caso de Uso 02 - Consultar Saldo

Descrição do Caso de Uso

Objetivo: Permitir que o usuário visualize o saldo disponível e um resumo rápido (total gastos no período, total receitas). Pré-condição: Usuário autenticado.

Classes Participantes

- **Usuario** — requisita a consulta.
- **Login** — autenticação.
- **Saldo** — fornece o valor atual.
- **RepositorioTransacao / RepositorioGasto** — leitura de transações para cálculo de resumo.
- **ServicoRelatorio** — agrega dados para apresentação.

Descrição da Colaboração (Fluxo Principal)

1. **Usuario** solicita "Consultar Saldo".
2. **Login** valida sessão.
3. **ServicoRelatorio** solicita **Saldo** e transações recentes aos repositórios.
4. Dados agregados e calculados (saldo, totalGastos, totalReceitas).
5. **ServicoRelatorio** retorna resultado à UI.

6. UI exibe o resumo ao **Usuario**.

Fluxos Alternativos

- A1: Sessão inválida → redirect para login.
- A2: Falha de leitura → exibir mensagem de erro e opção de tentar novamente.

Diagrama de Sequência (descrição textual)

```
Usuario → Sistema/UI : solicitarConsultarSaldo()
Sistema/UI → ServicoAutenticacao : validarSessao()
ServicoAutenticacao → ServicoRelatorio : autorizar()
ServicoRelatorio → RepositorioSaldo : obterSaldo(usuarioId)
RepositorioSaldo → ServicoRelatorio : saldoAtual
ServicoRelatorio → RepositorioGasto : obterTotais(periodo)
RepositorioGasto → ServicoRelatorio : totais
ServicoRelatorio → Sistema/UI : retornarResumo(saldo, totais)
Sistema/UI → Usuario : exibirResumo()
```

Resultado Esperado

Exibição do saldo atual e resumo das transações.

Diagrama de Classes Participantes

(Adicionar no Astah: **Usuario**, **Login**, **Saldo**, **RepositorioSaldo**, **ServicoRelatorio**, **RepositorioGasto**.)

Caso de Uso 03 - Cadastrar Categoria

Descrição do Caso de Uso

Objetivo: Permitir que o usuário crie/edite categorias de gastos para organizar despesas. Pré-condição: Usuário autenticado; permissões para gerenciar categorias (se aplicável).

Classes Participantes

- **Usuario** — inicia a criação/edição.
- **Categoria** — entidade de domínio.
- **RepositorioCategoria** — persiste categorias.
- **ServicoCategoria** — regras de negócio (ex.: nome único por usuário).

Descrição da Colaboração (Fluxo Principal)

1. **Usuario** abre tela de gerenciamento de categorias.
2. Usuário escolhe "Criar Categoria" e fornece nome, percentual (opcional).
3. **ServicoCategoria** valida unicidade e regras.
4. **RepositorioCategoria** persiste a nova **Categoria**.
5. UI confirma operação.

Fluxos Alternativos

- A1: Nome de categoria duplicado → solicitar outro nome.
- A2: Falha de persistência → informar erro.

Diagrama de Sequência (descrição textual)

```

Usuario → Sistema/UI : solicitarCriarCategoria(dados)
Sistema/UI → ServicoCategoria : validarCategoria(dados)
ServicoCategoria → RepositorioCategoria : salvar(categoria)
RepositorioCategoria → ServicoCategoria : confirmacaoSalvar
ServicoCategoria → Sistema/UI : confirmarCriacao()
Sistema/UI → Usuario : exibirMensagemSucesso()
  
```

Regras de Negócio

- Nome da categoria deve ser único por usuário (ou por sistema, conforme regra).

- Percentual, se informado, deve estar entre 0 e 100.

Resultado Esperado

Categoria criada/atualizada e disponível para associação em gastos.

Diagrama de Classes Participantes

(Adicionar no Astah: `Usuario`, `Categoria`, `RepositorioCategoria`, `ServicoCategoria`.)

Caso de Uso 04 - Autenticar (Login)

Descrição do Caso de Uso

Objetivo: Autenticar o usuário no sistema para permitir acesso a funcionalidades protegidas. Pré-condição: Usuário cadastrado.

Classes Participantes

- `Usuario` — fornece credenciais.
- `Login` — entidade/serviço de autenticação.
- `RepositorioUsuario` — armazena credenciais/dados de usuário.
- `ServicoAutenticacao` — compara credenciais, gerencia tokens/sessões.

Descrição da Colaboração (Fluxo Principal)

1. `Usuario` envia credenciais (email/username + senha).
2. `ServicoAutenticacao` consulta `RepositorioUsuario`.
3. Se credenciais válidas, serviço emite token de sessão (ou cookie) e retorna sucesso.
4. Caso inválido, mensagem de erro retornada.

Fluxos Alternativos

- A1: Usuário não cadastrado → instruir para cadastro.
- A2: Senha incorreta → exibir erro e opção de recuperar senha.

Diagrama de Sequência (descrição textual)

```
Usuario → Sistema/UI : enviarCredenciais()
Sistema/UI → ServicoAutenticacao : autenticar(credenciais)
ServicoAutenticacao → RepositorioUsuario :
consultarUsuario(identificador)
RepositorioUsuario → ServicoAutenticacao :
dadosUsuario(hashPwd)
ServicoAutenticacao → ServicoAutenticacao :
verificarSenha(hashPwd, senhaFornecida)
ServicoAutenticacao → Sistema/UI : emitirToken/session()
Sistema/UI → Usuario : redirecionarParaDashboard()
```

Regras de Negócio

- Senhas devem ser armazenadas apenas em forma hashed+salt.
- Limitar tentativas de login e aplicar bloqueio temporário se necessário.

Resultado Esperado

Sessão autenticada e token/sessão criada para o usuário.

Diagrama de Classes Participantes

(Adicionar no Astah: `Usuario`, `Login/ServicoAutenticacao`, `RepositorioUsuario`.)

Caso de Uso 05 - Gerar Relatório/Sumário

Descrição do Caso de Uso

Objetivo: Gerar relatórios ou resumos (mensal, por categoria, por conta) para o usuário analisar suas finanças. Pré-condição: Usuário autenticado.

Classes Participantes

- **Usuario** — solicita o relatório.
- **ServicoRelatorio** — reúne e processa os dados.
- **RepositorioGasto / RepositorioReceita / RepositorioSaldo** — fontes de dados.
- **Relatorio** (objeto) — modelo de saída (JSON/PDF/CSV).

Descrição da Colaboração (Fluxo Principal)

1. **Usuario** solicita geração do relatório, informando parâmetros (período, tipo).
2. **ServicoRelatorio** consulta repositórios necessários.
3. Dados agregados e aplicadas regras de apresentação (agrupamentos, totais, percentuais).
4. **Relatorio** é gerado (formato escolhido) e disponibilizado para download/exibição.

Fluxos Alternativos

- A1: Período inválido → solicitação de ajuste.
- A2: Volume de dados grande → executar geração em background e notificar quando pronto.

Diagrama de Sequência (descrição textual)

```

Usuario → Sistema/UI : solicitarRelatorio(parametros)
Sistema/UI → ServicoRelatorio : gerarRelatorio(parametros)
ServicoRelatorio → RepositorioGasto : obterGastos(periodo)
RepositorioGasto → ServicoRelatorio : listaGastos
ServicoRelatorio → RepositorioReceita : obterReceitas(periodo)
ServicoRelatorio → ServicoRelatorio : agregarDados()
ServicoRelatorio → Sistema/UI : retornarRelatorio(relatorio)
Sistema/UI → Usuario : exibirOuDownload(relatorio)
  
```

Regras de Negócio

- Relatórios respeitam permissões do usuário.
- Opcional: geração assíncrona para períodos extensos.